FusionML: Prefill, Not Decode — Mechanism and Boundaries of CPU+GPU Co-Execution on Unified-Memory Apple Silicon

Om Mohite [1]

Abstract

Apple-Silicon SoCs place CPU, GPU, and Neural Engine on one unified memory system, inviting the question of whether transformer inference can be accelerated by executing single operators cooperatively across units. Prior attempts (including our own) failed or produced precision-confounded wins. We identify the mechanism behind these failures: MLX, the platform's lazy-graph array framework, *serializes* cross-stream work whenever a CPU-stream operation consumes an unmaterialized GPU result inside one evaluation graph — a row-split matmul that runs $1.38\times$ faster with materialized inputs runs $0.66\times$ (slower than GPU-only) inside a lazy graph. Inserting an eager materialization boundary before each split restores true concurrency ($1.34\times$). Building on this, FusionML implements per-layer, contention-aware CPU+GPU row splitting for transformer prefill, evaluated under a strict fairness protocol (precision-matched FP16 baselines, adjacently measured, with power and thermal state recorded in every result). Across five chips spanning three Apple-Silicon generations, community-replicated, the split accelerates Llama-shaped decoder-block prefill by $1.15\times$–$1.38\times$; the win is unchanged at full 32-block model depth (15.6 GB), and reaches end-to-end reality as $1.18\times$–$1.25\times$ faster time-to-first-token for a real Qwen2.5-7B checkpoint served through stock MLX-LM, with token-identical outputs in 5 of 6 configurations and unchanged decode throughput. We characterize the boundaries with equal care: decode cannot benefit (it is bound by the *shared* memory bandwidth co-execution does not add); single-block training loses $0.86\times$–$0.97\times$ on all five chips once the baseline is precision-matched; Neural-Engine dispatch overhead ($\sim$20 ms via CoreML) excludes it at layer granularity; and a runtime probing gate that guarantees no regression in-regime becomes *destructively self-deceiving* under memory pressure, where probing an alternative execution mode evicts the active mode's working set and biases its own measurement. We release all code, raw results, and transcripts.

1 Introduction

Unified-memory SoCs are now the dominant local-inference platform, and Apple Silicon is their most widely deployed instance. Because CPU, GPU, and Neural Engine (ANE) address the same physical memory, work can in principle be split *within* a single operator — rows of one matmul computed simultaneously on CPU and GPU — without any copy. Whether this helps in practice has remained unclear: published speedups in this space are frequently confounded (FP16 systems compared against FP32 baselines), and naive implementations lose outright.

This paper reports what we believe is the first mechanism-level account of when intra-operator CPU+GPU co-execution helps transformer inference on this platform, verified end-to-end and replicated by community contributors on five chips. Our contributions:

1. **Mechanism.** We show that lazy-graph evaluation is the hidden serializer: MLX runs CPU-stream and GPU-stream work concurrently only when the CPU operation's inputs are already materialized. A minimal probe (Section 3, Figure 1) makes the effect quantitative ($1.38\times$ ready / $0.66\times$ lazy / $1.34\times$ eager-boundary) and explains a history of failed co-execution attempts, including why an eager per-layer Swift implementation succeeded where lazy Python graphs failed.

2. **Design.** FusionML: a per-layer row-split for prefill (Figure 2) with (i) eager materialization boundaries, (ii) per-shape ratios calibrated *under contention* rather than from solo-unit rates, and (iii) a runtime gate (probation + periodic re-probing + hysteresis) that holds a no-regression floor by construction, since the baseline execution mode is itself one of the gate's arms (Section 4).

3. **Fair, replicated evaluation.** Under a strict protocol (Section 5.1): $1.15\times$–$1.38\times$ Llama-shape prefill wins

[1] Independent Researcher, Mumbai, India.

on all five chips tested (M1, M2, M3 Pro, M4, M4 Pro); unchanged at full 32-block depth; $1.18\times$–$1.25\times$ TTFT on a real 7B checkpoint through stock MLX-LM with token-identical outputs and neutral decode (Section 5).

4. **Boundaries, measured.** Decode is structurally out of scope on shared bandwidth; precision-matched training loses on every chip; ANE dispatch overhead and three (since OS-fixed) crash conditions exclude the Neural Engine at layer granularity; and we quantify an *observer effect*: under memory pressure, a probing scheduler's measurements of alternative modes are biased $\sim 1.4\times$ by the probe's own working-set eviction, with $\sim 1.7\times$ reload cost on the next call (Section 6).

We highlight a methodological stance: every headline number in this paper is measured against a *precision-matched* baseline executed adjacently on the same machine, and several of our own earlier "wins" are retired in Section 6 as FP16-vs-FP32 artifacts. Raw result files — including power source, battery level, thermal notes, and free memory at run start — accompany every experiment in the repository.

2 Background and Related Work

Apple Silicon and MLX. Apple's M-series SoCs share one LPDDR memory system across CPU (with AMX matrix units reachable via Accelerate), GPU (Metal), and ANE (reachable only through CoreML). MLX (Hannun et al., 2023) is Apple's array framework for this platform: lazily evaluated, with a `stream` abstraction that places operations on CPU or GPU. MLX-LM is its de-facto LLM serving layer, our end-to-end baseline.

Heterogeneous mobile inference. CoDL (Jia et al., 2022) co-executes CNN operators across CPU and GPU on smartphones and reports the same class of contention effects we measure (shared-bandwidth degradation of per-unit throughput); we extend the contention-aware view to LLM prefill on Apple Silicon and add the lazy-graph serialization mechanism, which has no analogue in eager mobile stacks. Phase-splitting systems such as Splitwise (Patel et al., 2024) and chunked-prefill schedulers such as Sarathi (Agrawal et al., 2023) exploit the prefill/decode asymmetry at cluster scale; we exploit the same asymmetry *within one SoC*.

Kernel- and block-level acceleration. FlashAttention (Dao et al., 2022) and successors accelerate attention itself; our split leaves attention on the GPU untouched and is complementary. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) accelerates decode with a draft model; we argue in Section 6 that a CPU/ANE-hosted draft is the natural way to use idle units during decode, precisely because intra-operator splitting cannot help there.

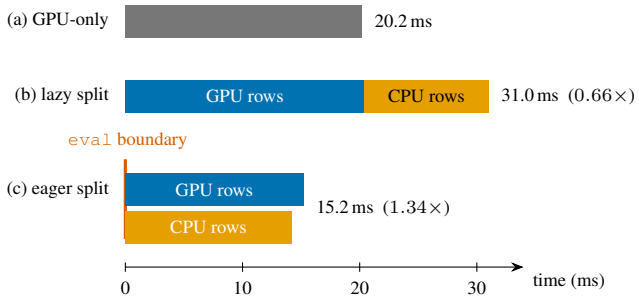

Figure 1. Execution timeline of one row-split matmul ($2048\times1600 \times 1600\times6400$, FP16, 35% CPU rows, M1; measured wall times). Inside one lazy graph (b) MLX serializes the streams and the split *loses*; an eager materialization boundary (c) restores concurrency.

3 The Serialization Mechanism

Row-splitting one matmul across `mx.cpu` and `mx.gpu` streams is straightforward; making it concurrent is not. Figure 1 shows the probe that isolates the effect. When both stream branches read an already-materialized array, MLX executes them concurrently and the split delivers the expected win ($1.38\times$). When the same split consumes the output of a GPU operation *within the same evaluation graph* — the situation of every layer after the first in any real network — execution serializes and the split is strictly worse than doing nothing ($0.66\times$). An explicit materialization boundary (`mx.eval` on the input) before the split restores concurrency at negligible cost ($1.34\times$).

This single scheduling property explains a striking prior split within our own project: an eager, per-layer Swift implementation showed consistent co-execution wins while every lazy-graph Python variant — including a coarse whole-FFN split whose CPU portion waited on attention — lost. The lesson generalizes: **lazy-graph frameworks cannot exploit intra-operator heterogeneous parallelism without eager boundaries**, an API-level consideration for any unified-memory array framework.

4 FusionML Design

Per-layer split. Every linear-layer matmul in a decoder block (QKV/output projections; gate/up/down or FC layers) is row-split: the first ρM rows execute on the CPU stream, the remainder on the GPU stream, concatenated afterward (Figure 2). Attention score/softmax/value math stays on the GPU. Each split is preceded by an eager boundary per Section 3.

Contention-aware calibration. The ratio ρ is selected per matmul shape by measuring the *actual concurrent split* over a small candidate grid ($\rho \in \{0, 0.10, \ldots, 0.50\}$), not by

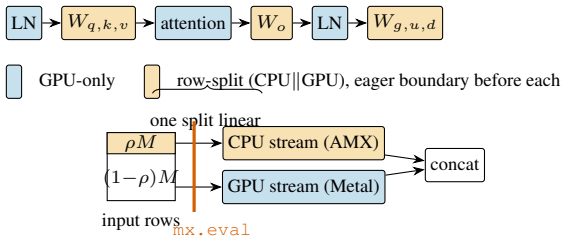

Figure 2. FusionML per-layer prefill split. Every linear layer is row-split across concurrent CPU/GPU streams behind an eager boundary; attention, normalization, and element-wise math stay on the GPU. The CPU share ρ is calibrated per matmul shape *under contention* ($\rho \approx 0.25$–0.40 across chips).

composing solo-unit throughputs: under co-execution, GPU throughput itself degrades (13.2 → 15.5 μs/row on M1) because the units share bandwidth, so solo rates systematically overstate the optimal CPU share.

No-regression gate. A runtime controller treats {split, compiled-baseline, eager-baseline} as competing execution modes: a short probation picks the fastest, and the inactive modes are re-probed periodically (every 10th call, exponential-moving-average tracking, 3% switching hysteresis). Because the stock execution path is itself an arm, the floor is the baseline minus bounded probe overhead ($\leq$3% on the smallest cells we measure, $<$1% typically). The eager-baseline arm is necessary: `mx.compile`d execution — FusionML's original default — itself loses $\sim$6% to eager MLX at sequence length 8192, so a gate over {split, compiled} alone has no true floor.

5 Evaluation

5.1 Setup and Fairness Protocol

Machines. Five chips across three generations, four of them run by community contributors from the public repository: M1 (8 GB, fanless MacBook Air), M2 (8 GB), M3 Pro (18 GB), M4 (24 GB), M4 Pro (24 GB); all on mains power.

Protocol. All comparisons are precision-matched (FP16 vs. FP16) and measured *adjacently* (baseline and treatment interleaved in the same session, one process at a time, cooldowns between subprocesses). n=50 timed runs after 10 warmups per cell (full-depth: n=10/3). Every result JSON embeds hardware identity, power source, battery level, thermal notes, and free memory at start; one contributed run that violated the memory requirement (a 15.6 GB model resident in 1.8 GB) was detected from these fields and excluded, and the harness now refuses such runs. Split outputs are verified against GPU-only execution (max relative error $\leq 3\times10^{-3}$ at FP16 accumulation, $\leq 3.9\times10^{-3}$ through 32 blocks).

5.2 Block-Level Prefill Across Five Chips

Figure 3: the split wins 1.15×–1.38× on Llama-shaped blocks on every chip with active cooling, replicated across independent sessions ($\pm$0.06 across two sessions on M4). Two honest qualifications. First, GPT-2-shaped blocks (d=1600) win less (1.02×–1.18×) and are noise-sensitive on 8 GB machines at short sequence lengths — short CPU chunks are hypersensitive to scheduler and thermal state. Second, we predicted win magnitude would track the chip's CPU:GPU core ratio; the prediction *partially failed* (M2 outperforms M4; M3 Pro trails both), and we report magnitude as chip-dependent. The robust cross-chip claim is the consistency of the win, not a single-variable law.

5.3 Full Model Depth

Stacking 32 blocks (15.6 GB of weights, per-block eager boundaries) leaves the win essentially unchanged (Figure 4: 1.12×/1.24×/1.20× at 2048/4096/8192 vs. 1.17×/1.25×/1.17× single-block). Per-layer synchronization cost does not compound with depth, and correctness holds through the full stack.

5.4 End-to-End: Real Checkpoint, Real Runner

We patch MLX-LM's `nn.Linear` so that large-row (prefill) calls use the split and decode calls (single-row) fall through untouched, and serve Qwen2.5-7B-Instruct-bf16 with greedy decoding. TTFT improves 1.18×–1.25× across 2k/4k/8k-token prompts on both 24 GB machines (Figure 5); decode throughput is exactly neutral, as designed. Outputs are token-identical to stock in 5 of 6 configurations; in the sixth, generation diverged at one near-tie token after 289 identical characters — FP16 reduction-order sensitivity, with both completions coherent. Verbatim prompt/response transcripts for every run ship with the repository. TTFT is the latency that dominates prefill-heavy workloads (RAG, document QA, long-prompt/short-answer), which is precisely the regime this mechanism targets.

5.5 The Gate, and the Observer Effect

In-regime, the gate behaves as constructed: on M4 it holds $\geq$1.0× in every cell, riding the split where it wins and standing down to baseline where it does not (e.g., 0.98× worst-case on a 15 ms cell where probe overhead is proportionally largest). Out of regime it taught us something better than success. On the 8 GB M1 at sequence length 8192 — where three execution modes' buffers cannot coreside — the gate correctly selected the fastest mode *by its own measurements* and still lost: each probe of an inactive mode evicted the active mode's working set, inflating the probe's reading $\sim$1.4× above that mode's true fresh-process latency and imposing $\sim$1.7× reload cost on the next active

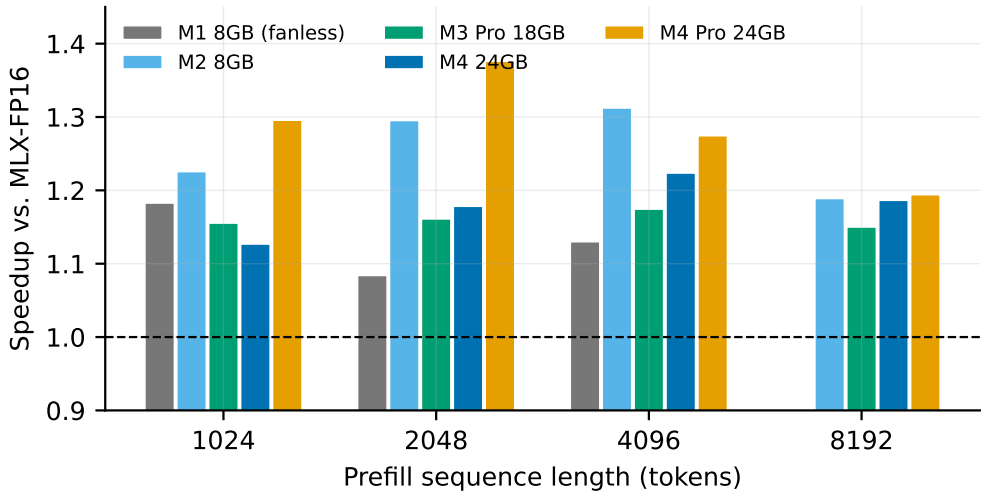

Figure 3. Llama-3-8B-geometry decoder-block prefill: per-layer split speedup over precision-matched MLX-FP16, five chips, sequence length 1024–8192. Every actively-cooled chip wins at every length; the fanless 8 GB M1 fades at 8192 under memory pressure and thermal throttling.

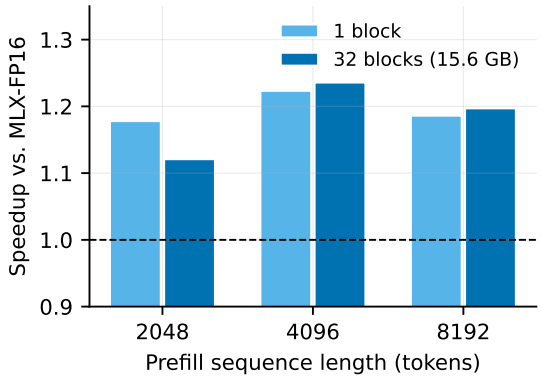

Figure 4. Single block vs. 32 stacked blocks (15.6 GB FP16 weights), M4 24 GB. Depth does not erode the win.

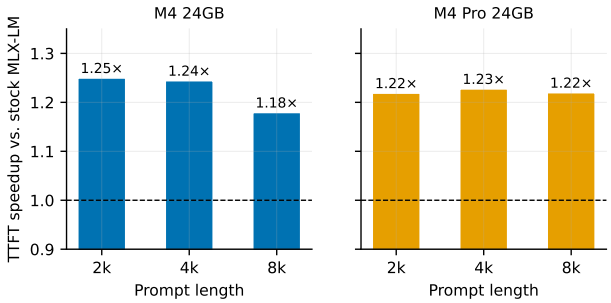

Figure 5. Time-to-first-token speedup for Qwen2.5-7B-Instruct (bf16) served through stock MLX-LM with FusionML's split patched into `nn.Linear` (prefill-scale calls only; untuned fixed ρ=0.30). Greedy decoding; decode throughput unchanged (7.1→7.1 tok/s on M4, 13.4→13.5 on M4 Pro).

call. A periodically-probing scheduler under memory pressure cannot observe its alternatives without damaging both the measurement and the system. The practical rule we adopt: continuous probing only where working sets co-fit; probation-then-lock with drift-triggered (not scheduled) re-probing otherwise.

6 Boundaries: Where Co-Execution Cannot Help

Decode. Each decoded token streams the full weight set through the memory system once; decode is bandwidth-bound, and unified memory means the bandwidth is *shared*. Co-execution adds compute, not bandwidth, so no intra-operator split can accelerate decode on this architecture — confirmed by our decode-neutral end-to-end measurements. The idle-unit opportunity during decode is speculative decoding with a CPU/ANE-hosted draft model, which spends idle *compute* to save *bandwidth per accepted token*; we leave it to future work.

Training (a retired claim). FusionML's earlier training "wins" (1.13×–1.35×) were measured against FP32 baselines while computing in FP16. Precision-matched, single-block training loses on *all five chips* (0.86×–0.97×), with finite losses verified. We report this as the strongest honest negative in the paper and a caution for the literature: of the headline comparisons we began with, every one that mixed precisions reversed or vanished when matched.

Neural Engine. ANE is reachable only through CoreML, whose dispatch costs $\sim$20–24 ms fixed plus $\sim$7 μs/row regardless of chip generation (measured M1 through M4 Pro) — useless at layer granularity where matmuls take 2–20 ms, though it amortizes at batch scale, where our contention-aware three-way search reached $1.61\times$–$1.79\times$ burst ($1.16\times$–$1.29\times$ sustained) on raw FFN-shaped matmuls. Three deterministic CoreML/MLX coexistence crashes we isolated on macOS 25.x (e.g., loading a second distinct compiled shape segfaults the process) are fixed in macOS 26.3 — verified by rerunning our minimal repros on two machines — but the dispatch overhead stands, and ANE weight-baking costs 1.4% relative error on routed rows.

Workload shape. Co-execution is not universal even at batch scale: a blocked Cholesky whose trailing update is SYRK-shaped never beat CPU-only Accelerate BLAS ($0.90\times$–$0.95\times$) — when a single unit's specialized path already saturates the bottleneck resource, adding units only adds contention.

Thermal envelope. On the one passively-cooled machine, sustained load reduces every speedup (burst $1.79\times \rightarrow$ sustained $1.16\times$–$1.29\times$ on raw matmuls) and small-shape results become session-dependent; we therefore report cold/hot states separately and treat fanless chassis as the motivating case for runtime adaptivity rather than a reliable benchmark platform.

7 LIMITATIONS

Our transformer measurements use decoder-block geometries with synthetic weights for controlled experiments (latency is weight-value-independent; correctness is separately verified) and one real 7B checkpoint end-to-end; broader model coverage (MoE, multimodal) is future work. The mechanism analysis is specific to MLX's scheduler, though we expect the eager-boundary requirement to generalize to any lazily-evaluated framework with device streams. Ratios in the end-to-end experiment are untuned (ρ=0.30 fixed); per-shape calibration should widen the reported TTFT wins. Batch serving throughput — where raw-matmul results suggest the largest headroom — is measured at the operator level but not yet end-to-end.

8 CONCLUSION

On unified-memory SoCs, intra-operator CPU+GPU co-execution is neither a myth nor a free lunch: it is a prefill-scoped mechanism, gated by an eager-boundary scheduling requirement that lazy-graph frameworks violate by default, bounded by shared bandwidth in decode, by dispatch overhead on the ANE, and by working-set co-residency for any scheduler that probes its alternatives. Within its regime it is real, replicated, and free: up to $1.38\times$ block-level and $1.25\times$ end-to-end TTFT on stock MLX-LM with token-identical outputs. We hope the fairness protocol — precision-matched adjacent baselines, environment-stamped raw results, and retired claims reported alongside surviving ones — is as useful to the community as the mechanism itself.

ACKNOWLEDGMENTS

Community contributors ran the M2, M3 Pro, M4 Pro, and portions of the M4 benchmark suites on their own hardware; they will be named (or credited as they prefer) in the camera-ready. Code, raw environment-stamped results, and verbatim generation transcripts: https://github.com/ommo007/FusionML.

REFERENCES

Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., and Ramjee, R. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills, 2023.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Hannun, A., Digani, J., Katharopoulos, A., and Collobert, R. MLX: Efficient and flexible machine learning on apple silicon. https://github.com/ml-explore/mlx, 2023.

Jia, F., Zhang, D., Cao, T., Jiang, S., Liu, Y., Ren, J., and Zhang, Y. CoDL: Efficient CPU-GPU co-execution for deep learning inference on mobile devices. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, 2022.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.